

Informatics II, Spring 2026, Exercise 10

Publication of exercise: May 11, 2026

Publication of solution: May 18, 2026

Exercise classes: May 18 - 22, 2026

Dynamic Programming

Task 1 (Medium – approx. 30 minutes)

A path in the 2-dimensional matrix M of dimensions $x \times y$ is defined a sequence of squares (i_1, j_1) , (i_2, j_2) , $\dots$, (i_n, j_n) , where $1 \leq i_k \leq x$ and $1 \leq j_k \leq y$ for all $k = 1, 2, \dots, n$, and the difference in value between consecutive squares in the path, $|M_{i_k, j_k} - M_{i_{k+1}, j_{k+1}}| \leq 1$ for all $k = 1, 2, \dots, n - 1$. The path must also satisfy the constraint that (i_{k+1}, j_{k+1}) can only be either directly below (i_k, j_k) or directly to the right of (i_k, j_k) , but not directly above or to the left of (i_k, j_k) . The longest possible path within the matrix is the one with the maximum value of n .

$$\begin{pmatrix} 1 & 7 & 3 & 2 & 6 & 1 \\ 2 & 5 & 4 & 5 & 9 & 3 \\ 6 & 3 & 2 & 6 & 6 & 6 \\ 8 & 7 & 5 & 4 & 8 & 7 \end{pmatrix}$$

Table 1: Example of the longest path problem with the solution of length 7

Table 2 shows the helper matrix for the given matrix above. The helper matrix denotes in each field the path with the maximum length up to this point. As you can see the longest path from the example of table 1 starts with a size of 1 and ends with a size of 7.

$$\begin{pmatrix} 1 & 1 & 1 & 2 & 1 & 1 \\ 2 & 1 & 2 & 3 & 1 & 1 \\ 1 & 1 & 2 & 4 & 5 & 6 \\ 1 & 2 & 1 & 2 & 1 & 7 \end{pmatrix}$$

Table 2: Helper matrix to the matrix in table 1

- a) Fill in the helper matrix in table 4 for the matrix in table 3. Try to already find a pattern on how you approach this problem step by step. Remember that from any location on the path to the next location you can only go down or to the right.

$$\begin{pmatrix} 4 & 6 & 7 & 8 \\ 3 & 2 & 3 & 8 \\ 8 & 6 & 6 & 7 \\ 9 & 5 & 2 & 5 \\ 8 & 4 & 3 & 2 \end{pmatrix}$$

Table 3: Matrix to task 1. a)

Table 4: Helper table to fill in for task 1. a)

- b) Determine recursive definition for helper matrix.
- c) Use a bottom up approach to write the function *int longestPath(int x, int y, int M[x][y])* in the **task_1_skeleton.c** file from Olat which returns the value of the longest path in matrix *M* of dimension $x \times y$.

Task 2 (Hard – approx. 50 minutes)

The goal of this exercise is to determine the minimum number of cuts to divide a given string S into substrings $(s_1, s_2, s_3, \dots, s_n)$ such that each substring is a palindrome. One cut splits one string into two nonempty substrings.

An example for this problem would be the string: "ABACDDC" which can be sliced into two halves: "ABA" and "CDDC" in one cut. Those two substrings are both palindromes, so the solution to this example is 1, since only one cut is required.

Another example is the string "ABBCDC" which can be cut into: "A", "BB" and "CDC" thus requiring 2 cuts in total.

Find the amount of cuts needed for the following strings yourself:

- "ERTTZUTT"
- "KLOOOLK"
- "ZTUOOLAEAL"
- Write a helper function `int isPalindrome(char X[], int i, int j)` in the file `task_2_skeleton.c` from Olat which returns 1 if the string X is a palindrome from index i to (and including) index j , and 0 if $X[i] \dots X[j]$ is not a palindrome.
- Now see if the function `findMinCutsRecursive(char X[], int i, int j)` runs with your helper function, then analyse the recursive approach with the following points:
 - Estimate the overall runtime complexity (no exact derivation required, simply how many recursive calls are made for a string of length n).
 - Looking ahead to the next task, what do you expect to change when using a dynamic programming approach?
- In the dynamic programming solution, a helper matrix is used to store whether a substring $X[i] \dots X[j]$ is a palindrome. The upper half of the matrix is filled from the back of the string to the front. Each entry in the matrix corresponds to a substring $X[i] \dots X[j]$ (i always on diagonal, j somewhere in the row to the right) and is set according to the following rules:
 - Every single character is a palindrome, so all entries with $i = j$ are **true** (1).
 - If $X[i] == X[j]$ and the substring between them is empty or already known to be a palindrome, then $X[i] \dots X[j]$ is also a palindrome. All other substrings are not palindromes (therefore entry in matrix = **false** (0)).

Using this procedure, fill in the following helper matrix for the string "ABBC".

Table 5: Helper matrix for the string "ABBC"

- g) In the dynamic programming solution, a helper array is used to store the minimum number of cuts required for substrings of X . The helper array is filled from right to left. Each entry at index i represents the minimum number of cuts needed to split the suffix $X[i] \dots X[n-1]$ into palindromic substrings.

The array is computed using the helper matrix as follows:

- If the substring $X[i] \dots X[n-1]$ is a palindrome, then no cut is required and the value is 0.
- Otherwise, all positions $j \geq i$ are considered where $X[i] \dots X[j]$ is a palindrome. For each such position, one cut is made after index j , and the total number of cuts is $1 + \text{helper_array}[j + 1]$.
- The minimum value over all valid positions j is stored in $\text{helper_array}[i]$.

Using this procedure and the helper matrix from the previous task, fill in the helper array for the string "ABBC" (length 4).

--	--	--	--

Table 6: Helper array for the string "ABBC"

- h) In `task_2_skeleton.c` write the C function `int findMinCuts(char X[], int n)` using the dynamic programming approach just discussed, where n denotes the length of the string X . A valid call is `findMinCuts("ABBCDC", 6)`, which should return 2.

Task 3 (Medium – approx. 40 minutes)

The goal of this exercise is to find the largest plus-shaped structure in a binary matrix M , which has dimensions of $x \times y$.

The plus-shaped structure is made from 1s and must be symmetrical both horizontally and vertically. There is a central point and the ones are evenly spaced in the north, south, east, and west direction from the center.

The area of the plus can therefore be expressed mathematically as $A = 4h + 1$, where h represents the length of each of the 4 half-arms of the plus and A is the number of 1s in the plus.

$$\begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Table 7: Example biggest plus with $h = 2$ and $A = 9$

To calculate the result we want to make use of 4 helper tables: top, bottom, left and right. Each table should at each entry mark the amount of 1s that are directly (in the example of the top helper matrix) above it. In table 6 you can see an example of what the top helper table for the matrix in table 5 looks like.

- a) Fill in the helper tables bottom, left and right in the same fashion as the example given in table 6.

0	0	0	1	0	0
0	1	0	2	0	0
1	2	1	3	1	1
2	3	0	4	0	2
3	0	0	5	1	0

Table 8: Helper table top

Table 9: Helper table bottom

Table 10: Helper table left

Table 11: Helper table right

- b) Write a C function *void makeHelper(int x, int y, int M[x][y], int top[x][y], int bottom[x][y], int left[x][y], int right[x][y])* in `task_3_skeleton.c` from Olat. This function should transform the 4 initially empty helper matrices according to the input matrix *M* with dimension *x* x *y* into the helper matrices you created in task 3.a).
- c) The goal now is to combine the knowledge of those helper matrices created in subtask a) and b) to find the biggest plus in a binary matrix *M*. Write the C function *int biggestPlus(int x, int y, int M[x][y])* which returns the value of the area of the biggest plus in the given matrix *M*.